Draw It or Lose It

CS 230 Project Software Design Template

By: Jacob Hough

2/21/2026

Version 2.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 4
Domain Model 4
Evaluation 5
Recommendations 5

Document Revision History

Version	Date	Author	Comments
1.0	01/24/2026	Jacob Hough	Initial creation of the software design document, including system architecture, domain model, platform evaluation, and recommendations for the Draw It or Lose It application.

Executive Summary

The Gaming Room has requested assistance in expanding their existing Android-based game, *Draw It or Lose It*, into a web-based application capable of supporting multiple platforms. The goal of this effort is to design a scalable and maintainable software solution that allows multiple teams and players to participate in a game session while ensuring that critical constraints (such as unique game and team names) are enforced consistently. Because the application is intended to operate in a distributed environment, careful consideration must be given to how shared data and system state are managed.

To address these needs, this software design proposes a server-side application built using Java and object-oriented design principles. The design leverages established software design patterns, including the Singleton pattern to ensure that only one instance of the game service exists in memory, and the Iterator pattern to efficiently manage and validate collections of games, teams, and players. A shared domain model consisting of Game, Team, and Player entities provides a clear and extensible structure that supports the client's functional requirements while remaining adaptable to future enhancements such as persistent storage and enhanced security.

The recommended next steps are to continue refining the prototype application, validate the design decisions with stakeholders at The Gaming Room, and evaluate deployment options suitable for a web-based environment. Once these steps are complete, the application can be prepared for broader platform support and future scalability as the user base grows.

Requirements

The following business and technical requirements have been identified for the Draw It or Lose It application:

- *The application must support one or more teams participating in a single game instance.*
- *Each team must be able to contain multiple players.*
- *Game names and team names must be unique to prevent conflicts when creating or joining games.*
- *Only one instance of the game service may exist in memory at any given time to manage shared state and identifier generation.*
- *The application must be designed to support deployment in a web-based, distributed environment.*
- *The system should be structured to allow future enhancements such as persistent storage, additional security features, and scalability improvements.*

Design Constraints

The *Draw It or Lose It* application is intended to operate in a web-based, distributed environment, which introduces several design constraints that directly influence how the software is developed and structured.

One primary constraint is the distributed nature of the application. Because users may access the system concurrently from different platforms, the application must carefully manage shared data and system state. To address this, a single centralized game service is used to coordinate game creation and identifier assignment. This ensures consistency across all active game sessions and prevents conflicting data from being created simultaneously.

Another important constraint is concurrency and shared state management. Multiple users may attempt to create games, teams, or players at the same time. To prevent duplicate names and identifiers, the

application must validate new entries against existing collections before creating them. This constraint influenced the decision to use controlled access through a single service instance and consistent validation logic when adding new entities.

The requirement that game and team names remain unique also impacts the application design. Each time a new game, team, or player is created, the system must search the current collection to confirm that the chosen name is not already in use. This constraint necessitates efficient and reliable traversal of in-memory collections to enforce uniqueness rules consistently.

Because the application is a prototype, all data is currently stored in memory rather than in a persistent database. While this simplifies development and testing, it limits long-term scalability and durability. The design accounts for this limitation by organizing the application around a clear domain model, allowing persistent storage solutions to be introduced later without requiring significant restructuring.

Platform compatibility is another design consideration. The application is written in Java to support deployment across multiple operating systems and environments. This choice supports the client's goal of serving multiple platforms while maintaining a consistent server-side implementation.

Finally, security considerations represent an additional constraint. While advanced authentication and authorization mechanisms are not implemented in this prototype, the design anticipates the need to protect game data and service endpoints in a production environment. The centralized service model supports future enhancements such as access control, request validation, and secure communication channels.

System Architecture View

The *Draw It or Lose It* application is designed using a simple client–server architectural approach appropriate for a web-based, distributed environment. In this prototype, the focus is on the server-side application logic, which manages game sessions, teams, and players through a centralized game service.

Client devices, such as desktop or mobile systems, would interact with the application through a web interface or API layer in a production environment. These client requests would be handled by the server-side application, which contains the core business logic and domain model. Communication between clients and the server would occur over standard network protocols, allowing the application to support multiple platforms without requiring platform-specific server implementations.

At this stage of development, all application data is maintained in memory on the server. The GameService class acts as a central access point for managing shared state, including active games and identifier assignment. While persistent storage and distributed components are not implemented in this prototype, the architecture is designed to allow these features to be added in the future with minimal changes to the overall structure.

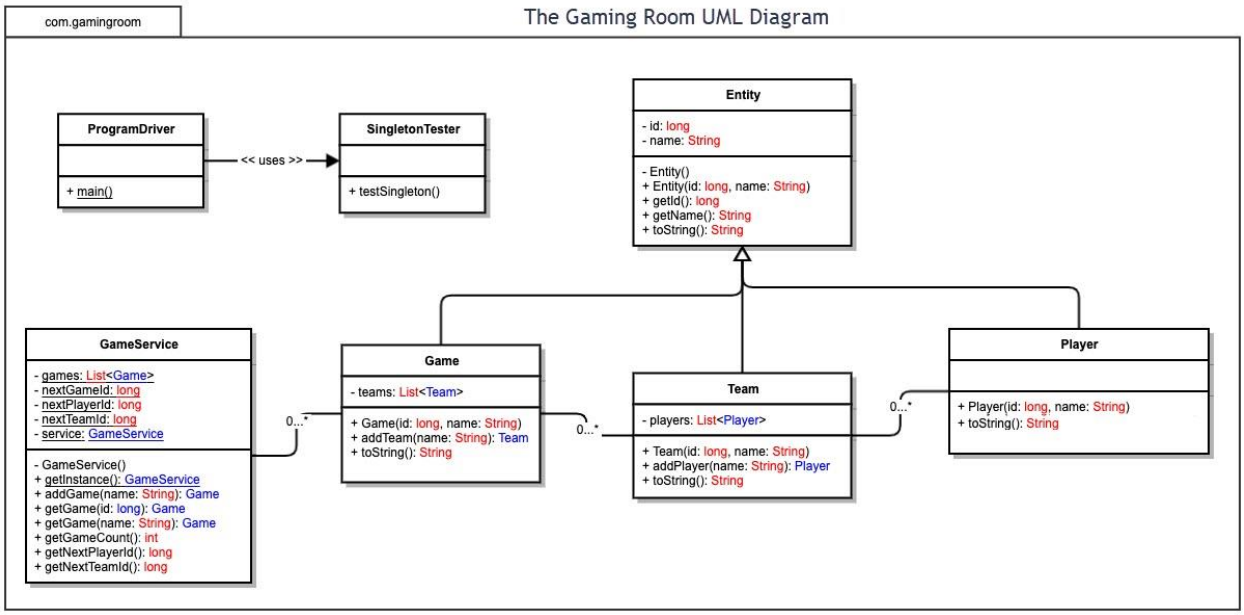
Domain Model

The domain model for *Draw It or Lose It* is organized around a small set of core entities that represent how the game is played. The Entity class serves as a shared base type that contains common attributes used throughout the system, including a unique identifier (id) and a name (name). The Game, Team, and Player classes inherit from Entity, which reduces duplication and ensures all major objects follow a consistent structure. A Game contains one or more Team objects, and each Team contains one or more

Player objects, reflecting the real-world rules of the application. The GameService class acts as the central access point for creating, managing, and retrieving active game instances. This structure ensures that games can support multiple teams and players while keeping the model easy to extend in future development phases.

This domain model supports the client's requirements by clearly separating responsibilities and enforcing consistency through shared behavior. Because each entity derives from Entity, the system can assign unique identifiers and manage names uniformly across games, teams, and players. The relationships between Game, Team, and Player directly support the requirement for multiple teams per game and multiple players per team. In addition, using a centralized GameService helps maintain a single authoritative source for game creation and lookup, reducing the risk of duplicate objects or conflicting identifiers. Overall, the domain model is designed to be readable, maintainable, and adaptable to future enhancements such as persistent storage or expanded game logic.

Inheritance is used by allowing Game, Team, and Player to extend the Entity class, which promotes code reuse and consistency. Encapsulation is achieved by organizing data and behavior within well-defined classes that manage their own state. Abstraction is represented by the Entity class, which defines common characteristics shared by all game-related objects while allowing specific implementations in derived classes. Polymorphism allows different entity types to be treated uniformly when appropriate, supporting flexibility and future expansion of the application. The domain model also demonstrates several object-oriented programming principles.



Evaluation

The development and deployment of the *Draw It or Lose It* application require consideration of multiple operating platforms for both development and hosting. Each platform offers different advantages depending on whether it is used for server-side deployment or client-side interaction.

Linux is well suited for server-side deployment due to its stability, performance, and widespread use in web hosting environments. It offers strong support for Java-based applications and is commonly used for scalable, distributed systems. Linux also provides flexibility for future enhancements such as containerization and cloud deployment.

Windows provides a user-friendly development environment with strong support for Java development tools and integrated development environments. It is commonly used by developers and is suitable for local development and testing of the application. While Windows can be used for server hosting, it is more commonly leveraged for development purposes.

macOS offers a stable development platform with excellent support for Java and cross-platform development tools. It is well suited for application development and testing, particularly in environments where developers may also target mobile or web-based clients.

Mobile platforms are primarily relevant on the client side rather than for server deployment. Mobile devices would access the application through a web interface or API layer, allowing the server-side application to remain platform-independent while supporting a wide range of client devices.

Overall, Linux is the most appropriate choice for server deployment, while Windows and macOS are effective platforms for development and testing. Mobile platforms are best supported through web-based access rather than direct deployment.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can host Java-based web applications and provides a stable environment for development and light server use. However, it is less commonly used for production server hosting compared to Linux and may involve higher hardware and licensing costs. macOS itself does not require a separate server license, but Apple hardware costs are typically higher than comparable Linux or Windows server hardware.	Linux is well suited for server-side deployment due to its stability, performance, and widespread use in web hosting environments. It provides strong support for Java applications and is commonly used for scalable, distributed systems. Most Linux distributions used for servers (such as Ubuntu Server) have no licensing cost, though enterprise support subscriptions (such as RHEL) can add optional costs.	Windows supports Java-based server applications and offers familiar administration tools. While it can be used for hosting, it is more commonly utilized for development environments rather than large-scale production servers. Windows Server typically requires paid licensing, and organizations may also incur additional costs depending on user access or enterprise features.	Mobile devices are not suitable for server-side hosting. They are intended for client-side access and interaction with the application through a web interface or API. Licensing costs are not applicable because mobile operating systems are not used for server deployment.

Client Side	macOS provides a strong development and testing environment for client-side access, particularly for web-based applications. It supports modern browsers and development tools used to test multi-platform clients. Because the client runs in a browser, compatibility testing is required across modern browsers (Chrome/Edge/Firefox/Safari) to ensure consistent behavior and performance.	Linux can support client-side access through web browsers and development tools. While less common for general users, it remains a viable platform for development and testing. Because the client runs in a browser, compatibility testing is required across modern browsers (Chrome/Edge/Firefox/Safari) to ensure consistent behavior and performance.	Windows is widely used by end users and provides strong client-side support through modern web browsers. It is well suited for accessing and testing the application from a client perspective. Because the client runs in a browser, compatibility testing is required across modern browsers (Chrome/Edge/Firefox/Safari) to ensure consistent behavior and performance.	Mobile devices are a primary client platform for accessing the application. Users would interact with the game through a web interface or API, allowing broad accessibility across different device types. Mobile support requires responsive design and testing on iOS Safari and Android Chrome to account for touch input, screen sizes, and mobile performance constraints.

Development Tools	Development tools such as Java, Eclipse, IntelliJ IDEA, and web browsers are well supported on macOS. These tools allow developers to build, test, and debug the application efficiently. Most tools are available on macOS, but using Xcode for native iOS work requires Apple hardware; additionally, some IDE editions (such as IntelliJ IDEA Ultimate) may involve licensing costs.	Linux supports a wide range of Java development tools, including Eclipse, IntelliJ IDEA, and command-line utilities. It is well suited for both development and server deployment workflows. Linux supports many free and open-source development tools, which can reduce licensing costs, but teams may need stronger command-line and server administration expertise.	Windows offers strong support for Java development through IDEs such as Eclipse and IntelliJ IDEA. It is commonly used for application development and testing due to its accessibility and tooling support. Windows supports common IDEs and tooling, but licensing costs may apply for Windows-based development environments and certain enterprise developer tools.	Mobile devices are typically not used to build the application directly; instead, development occurs on desktops using web tools, with validation focused on mobile browser compatibility and responsive design. If the client later chooses native mobile apps, iOS development would require Xcode on macOS and Android development would use Android Studio, which may increase time and team specialization.

Recommendations

Operating Platform

To support the expansion of *Draw It or Lose It* across multiple computing environments, the recommended server operating platform is **Linux**, specifically a distribution such as **Ubuntu Server**.

Linux is widely used in enterprise and cloud environments due to its stability, scalability, and security. It supports distributed systems, web-based applications, and cloud deployments efficiently. Because *Draw It or Lose It* must support Android, iOS, Windows, and web-based access, Linux provides a flexible and platform-neutral backend environment.

Additionally, Linux integrates well with cloud providers such as AWS and Microsoft Azure, making it easier for The Gaming Room to scale the application as user demand increases.

Operating System Architectures

Linux primarily uses a **monolithic kernel architecture**. In this architecture, core operating system services such as process management, memory management, device drivers, and file systems operate within the kernel space.

Key architectural characteristics include:

- **Process Management**: Linux supports multitasking and multi-threading, allowing multiple game sessions to run simultaneously without interference.
- **User Space vs. Kernel Space**: Applications like *Draw It or Lose It* operate in user space, while core OS operations run in kernel space. This separation improves stability and security.
- **Modular Design**: Although Linux uses a monolithic kernel, it supports loadable kernel modules. This allows components such as drivers to be added or removed without rebuilding the entire system.
- **Symmetric Multiprocessing (SMP)**: Linux efficiently distributes workloads across multiple CPU cores, which is important when handling many simultaneous players.

This architecture supports high concurrency, strong performance, and stability, all of which are essential for a real-time multiplayer game.

Storage Management

For storage management, a **relational database management system (RDBMS)** such as **MySQL** or **PostgreSQL** is recommended.

These systems provide:

- Structured data storage for user accounts, teams, scores, and game sessions
- ACID compliance to ensure data integrity
- Transaction management to prevent corruption during concurrent access
- Backup and recovery tools

On the file system level, Linux commonly uses **ext4**, which provides journaling capabilities. Journaling improves reliability by tracking changes before they are committed, reducing the risk of data corruption during unexpected shutdowns.

For scalability, storage can also be implemented using cloud-based database services, allowing vertical and horizontal scaling as the number of users increases.

Memory Management

Linux uses several advanced memory management techniques that benefit *Draw It or Lose It*:

Virtual Memory

Linux uses virtual memory to abstract physical RAM. Each process is given its own virtual address space, preventing applications from interfering with one another. This increases system stability and security.

Paging

Memory is divided into fixed-size pages. When RAM is limited, inactive pages can be moved to swap space on disk. This ensures that active game processes remain responsive.

Demand Paging

Memory pages are only loaded when required, which improves efficiency and reduces unnecessary memory consumption.

Memory Protection

Linux enforces memory protection between processes. If one process crashes, it does not impact others.

Garbage Collection (Application-Level)

If the backend is developed in Java (as discussed in earlier project phases), the Java Virtual Machine (JVM) manages heap memory using automatic garbage collection. This reduces memory leaks and improves long-term stability.

These memory management techniques ensure efficient handling of multiple simultaneous game sessions while minimizing crashes and performance degradation.

Distributed Systems and Networks

To allow *Draw It or Lose It* to communicate across Android, iOS, Windows, and web platforms, a **client-server distributed architecture** is recommended.

Architecture Overview

- Clients (mobile apps, web browsers, desktops) send requests to a centralized Linux-based server.
- The server processes game logic, manages sessions, and stores data in the database.
- Communication occurs over standard network protocols such as HTTP/HTTPS.

Networking Components

- **TCP/IP Protocol Suite** ensures reliable communication between devices.
- **RESTful APIs** allow platform-independent communication.
- **Load Balancers** distribute traffic across multiple servers.
- **Redundancy and Failover Systems** reduce downtime.

Dependency Considerations

- Network connectivity is required for real-time gameplay.
- Server outages would interrupt service; therefore, cloud redundancy is recommended.
- Latency must be minimized for fair gameplay timing.

Using a distributed architecture ensures scalability, reliability, and platform independence.

Security

Security must protect user data both at rest and in transit.

Data in Transit

- Use **HTTPS (TLS encryption)** to secure communication between clients and servers.
- Prevent interception through encrypted network channels.

Data at Rest

- Encrypt sensitive user information within the database.
- Use secure hashing algorithms (e.g., bcrypt) for passwords.

Authentication and Authorization

- Implement secure login systems.
- Use token-based authentication (such as JWT).
- Apply role-based access control to limit administrative privileges.

Operating System Security Features

Linux provides:

- User and group permission controls
- File access restrictions
- Firewall configuration (iptables or ufw)
- Regular security updates and patch management

Additional Protections

- Regular database backups
- Monitoring and logging for suspicious activity
- Protection against denial-of-service (DoS) attacks

By implementing these measures, The Gaming Room can ensure confidentiality, integrity, and availability of user data across platforms. Overall, the Linux-based distributed architecture provides scalability, reliability, security, and cross-platform compatibility, making it the most appropriate solution for The Gaming Room's long-term growth.